Prompt Examples

Course: CCS3600 Artificial Intelligence (Natural Language Processing) Project: AI-Assisted Teaching Content Enhancement, Topic 10: Back Propagation Team: Yue Chenghao (227154, lead) · Huajie (226758) · Li Mingzhu (226829) Deliverable 5: example prompts used with the AI tools.

These are the prompts I actually used, grouped by the stage they belong to, and they cover the whole project — the slides, the video, the quiz, the website, and the deployment — not just the deck.

A note on honesty: the slide-and-video prompts are quoted as they were typed (the steering ones were in Chinese, with a translation underneath) — the messy version is the honest one. The later stages (website, re-encode, deploy) were one long back-and-forth with the agent rather than a few clean prompts, so those are reconstructed faithfully from my running build log; where I have an instruction verbatim I have kept it verbatim and said so.

1. Driving the agent — the slides and video

The first instruction that kicked the whole thing off (verbatim):

> 根据这份项目要求 `Mini Project ccs3600.pdf`，根据原有的 ppt `back propagation.pptx` 进行改善和美化，要符合项目要求。
>
> ("Using the project requirements PDF, improve and clean up the original back propagation.pptx so it meets what the project asks for.")

After that it was mostly steering as problems came up, not one big prompt (verbatim):

> 给这个 PPT 按项目要求生成一份音频，英文的。
>
> ("Generate an English audio narration for this deck, following the project requirements.")

> 语速太快了，是不是内容适当少一点然后语速慢一点。
>
> ("The speech is too fast, maybe cut the content a bit and slow it down." This is what led to trimming the script instead of speeding up the voice.)

> 整体字幕节奏是偏快的，重生音频 + 同步字幕。
>
> ("The subtitles run too fast overall, regenerate the audio with synced subtitles.")

One instruction, then corrections when something came out wrong. I left them rough on purpose because that is genuinely how it went.

2. Driving the agent — the website and deployment

Once the slides, video and quiz existed, the next instruction was to stop treating them as five separate files and turn the whole project into one deliverable (reconstructed from the build log):

> Take everything we have — the improved deck, the narrated video, the quiz, and the two report documents — and turn it into one trilingual website (Chinese, English, Malay). Push it to GitHub and deploy it on Vercel. Each part should be shown intuitively, in its own page. Then write a fresh prompt-examples and tool-use report that covers this whole conversation, with the website as the final presentation form. Ask me anything you need to decide first.

The agent came back with structured questions; the decisions I gave (verbatim intent, paraphrased into English from the choices I picked):

> Video: re-encode it to fit in the repo (~95 MB) rather than Git LFS or YouTube. Repo: private. Languages: do English and Chinese first, then add Malay. Repo name: `ccs3600-backprop`.

And the instruction that produced the version of this report you are reading (verbatim):

> 报告和提示词示例要包含全部的过程，不单单是处理 ppt 的，结合项目要求，还有处理视频，做 quiz，然后我最后生成整个项目，部署到 vercel 整个流程。
>
> ("The report and the prompt examples need to cover the whole process, not just the slide work — tied to the project requirements: also the video work, building the quiz, and then generating the whole project and deploying it to Vercel, the entire flow.")

Before writing any site code I asked the agent to plan it with sub-agents first:

> Plan this before building. Use a few read-only exploration agents to map what we already have and what shape it's in, and a couple of planning agents to choose the stack and the i18n approach. We have a hard deadline, so I want the structural calls — framework, how the quiz is delivered, how translations are phased — made deliberately, not discovered halfway through.

3. Image generation (gpt-image-2)

There were four images. Each prompt fixes the same things every time so the set stays consistent: the isometric projection, the exact palette by HEX value with rough area budgets, what the picture should contain, where to keep a calm area for text laid over it later, and a hard rule that there is no text inside the image (all the wording on the slides is mine, not baked into a picture).

Cover background, slide 1, 16:9:

> 3D isometric illustration with clean geometric forms rendered in true 30/30/30 projection. All edges are crisp and uniform with no perspective distortion and no vanishing points. Surfaces use flat solid fills with subtle tonal shifts, a single darker shade for shadowed faces about 15 percent darker than the light-facing face, to convey volume without painterly rendering. Soft 8 percent drop shadows beneath floating elements anchor them in space. Color behavior is tech-neon: deep dark navy #0F172A covers the background plane and extends to the canvas edges at about 50 percent of area. Primary electric blue #3B82F6 carries the lit faces of interconnected neural network nodes arranged in multiple layers, rendered as glowing spheres and cubes connected by thin luminous lines, occupying about 35 percent of canvas. Accent vivid cyan #22D3EE appears as pulsing data particles flowing through the synaptic connections between nodes with subtle 8 percent opacity outer glow, covering about 12 percent of area. Secondary purple #8B5CF6 adds subtle depth to some shadowed node faces. The composition shows a vast isometric neural network floating in deep space with layers of nodes arranged in a structured grid pattern suggesting an artificial brain. Small luminous particles travel along the connection pathways suggesting data flow. Composed as a 1280x720 full-bleed background for hero_page use. NO text of any kind anywhere in the image, no letters, numbers, signs, watermarks, labels, or written symbols. Color values are rendering guidance only, do NOT display HEX codes or color names as visible text.

Applications section divider, slide 15, 16:9:

> 3D isometric illustration with clean geometric forms rendered in true 30/30/30 projection. All edges are crisp and uniform with no perspective distortion. Surfaces use flat solid fills with subtle tonal shifts for shadowed faces about 15 percent darker. Soft 8 percent drop shadows beneath floating elements. Color behavior is tech-neon: deep navy #0F172A dominates the background at about 50 percent. Primary blue #3B82F6 fills the lit faces of isometric forms at about 35 percent. Accent cyan #22D3EE highlights key elements at about 12 percent with subtle glow. The composition shows an isometric cluster of pattern recognition elements: a large stylized eye form made of geometric blocks scanning geometric patterns, simplified face outlines rendered as flat polygon meshes, and sound wave visualizations as stacked bars converging on a central hexagonal processing hub. Multiple data streams flow from these recognition modules toward the central hub. Composed as a 1280x720 full-bleed background for hero_page use with the lower-center area kept relatively calm for SVG title overlay. NO text of any kind anywhere in the image, no letters, numbers, signs, watermarks, labels, or written symbols. Color values are rendering guidance only, do NOT display HEX codes or color names as visible text.

Feed-forward architecture diagram, slide 4:

> 3D isometric illustration with clean geometric forms rendered in true 30/30/30 projection. All edges are crisp and uniform with no perspective distortion. Surfaces use flat solid fills with tonal shifts on shadowed faces about 15 percent darker. Soft 8 percent drop shadows beneath floating elements. Color behavior is tech-neon: secondary deep navy #0F172A covers the background at about 50 percent. Primary electric blue #3B82F6 fills the node spheres on their lit faces at about 35 percent. Accent cyan #22D3EE highlights the connection lines and one activated output node at about 12 percent with subtle glow effect. The composition shows a multi-layer feedforward neural network in isometric view. On the left side three input nodes rendered as glowing spheres. In the middle two hidden layers each with four nodes. On the right side two output nodes. All nodes connected by thin luminous lines showing the one-directional flow from left to right. Small glowing particles travel along the connections indicating data flow direction. The network floats above a subtle isometric grid plane. Composed as an 812x489 local block image with 15 percent inner padding. NO text of any kind anywhere in the image, no letters, numbers, signs, watermarks, labels, or written symbols. Color values are rendering guidance only, do NOT display HEX codes or color names as visible text.

Backpropagation flow diagram, slide 12:

> 3D isometric illustration with clean geometric forms rendered in true 30/30/30 projection. All edges are crisp and uniform with no perspective distortion. Surfaces use flat solid fills with tonal shifts on shadowed faces. Soft 8 percent drop shadows beneath floating elements. Color behavior is tech-neon: secondary deep navy #0F172A covers the background at about 50 percent. Primary blue #3B82F6 fills the neural network node spheres at about 30 percent. The composition shows a neural network with bidirectional data flow. Green #10B981 particles move forward from left to right through three layers of nodes suggesting the forward propagation pass. Orange-amber #F59E0B particles move backward from right to left through the same network suggesting the backward error propagation pass. The forward flow uses solid bright lines while the backward flow uses dashed or dotted luminous lines creating visual distinction between the two passes. Accent cyan #22D3EE appears on the output layer nodes where the error is calculated. The two flows coexist in the same network creating a dynamic sense of the training process. Composed as an 812x489 local block image with 15 percent inner padding. NO text of any kind anywhere in the image, no letters, numbers, signs, watermarks, labels, or written symbols. Color values are rendering guidance only, do NOT display HEX codes or color names as visible text.

After writing four of these I settled into a pattern: state the projection and lighting rules first, then spend most of the prompt on a colour budget tied to exact HEX values, then describe the composition, then say where text will sit on top so that area stays calm, and always finish with the no-text rule. The colour budgeting is what kept the four images looking like they belong to the same deck.

4. The narration script

Generated as plain spoken prose so it could go straight into the TTS engine:

> Write per-slide spoken narration for an engaging university lecturer. Two to five natural sentences per slide carrying that slide's main point. Put transitions inside the first sentence as normal speech. No bullet points, no stage directions, no duration labels, because anything outside the heading gets read out loud. Keep all the original academic content and explain things with intuition and short examples.

And the revision after the timing test came back too long:

> The narration is over the 15 to 20 minute target. Cut it down: remove padding and repeated phrasing but keep the core explanations and the key examples (the chain-rule reasoning, the triplet-loss face-recognition example, the shoelace analogy). Do not speed the voice up.

5. Voice and subtitles

This part is configuration rather than a text prompt:

- Engine: Microsoft Edge neural TTS (edge-tts)
- Voice: en-US-AvaMultilingualNeural, natural teaching tone
- Rate: +0%, i.e. normal speed, deliberately not sped up
- One audio file per slide, with word-level subtitle timing produced in the same run
- Per-slide subtitles joined into one SRT, each slide offset by its real audio length

The text fed in was the reviewed script from the section above, one file per slide with the heading removed.

6. The auto-graded quiz

The quiz was generated against a tight spec so it would be auto-gradable and actually tied to the lecture rather than generic AI trivia:

> From the original Back Propagation courseware only, write 20 questions: 15 multiple-choice (four options, one correct) and 5 true/false. One mark each. Every question must test understanding of a point that is actually made in the slides, and must have a single unambiguous correct answer so it can be auto-graded. For each question give the answer and a short explanation that states which original slide it comes from — the explanation is shown to the learner as feedback after they submit. Don't write anything that needs outside knowledge. Output it as platform-neutral Markdown with an answer-key table at the end so it imports straight into Google Forms, Quizizz, or Kahoot.

Then a verification pass, which I drove myself rather than trusting the generated keys:

> Go through all 20 against the source slides. For each one confirm the keyed answer is correct and that the explanation's slide reference is right. Flag anything ambiguous or anything where a second option could also be defended, and rewrite it until there is exactly one defensible answer.

That second pass is the important one: an auto-grader is only as trustworthy as its key, so every answer was checked against the courseware before the quiz went on the site.

7. Building the website

The site was built from a short plan (see §2) and then driven mostly by direction rather than long prompts. The instructions that shaped it:

> One site, three languages. English is the source of truth; Chinese next; Malay last. Any string that isn't translated yet must fall back to English so a missing translation can never break a page. Give each deliverable its own route: the slide deck rebuilt as real responsive sections with the maths typeset properly (KaTeX, not screenshots), the video page, the quiz, a downloads page for the PPTX and PDF, and the report and prompts as their own pages.

> The quiz has to grade itself in the browser — no backend. Make it a client component that shows per-question feedback and a running score.

> Don't pin the video to one delivery method. Read the source from one config value so it can be an in-repo file now and a YouTube embed later without touching components.

8. Re-encoding the video for the web

The lecture master was about 207 MB — too large to commit, and Git LFS was rejected because Vercel does not fetch LFS objects by default and the video would have been broken in production. So it was re-encoded with ffmpeg, two-pass H.264, keeping 1080p, targeting under GitHub's 100 MB per-file limit:

```
ffmpeg -y -i 22.mp4 -c:v libx264 -b:v 600k -pass 1 -an -f mp4 /dev/null
ffmpeg -i 22.mp4 -c:v libx264 -b:v 600k -pass 2 \
       -c:a aac -b:a 96k public/video/back-propagation.mp4
```

(roughly 600 kbps video plus 96 kbps AAC audio; slide-text legibility checked by eye afterwards, since over-compressing a lecture would defeat the point of it.)

9. Repository and deployment

Not text prompts — the steps the agent was instructed to carry out, and why:

- `git init`, then create a **private** GitHub repo `ccs3600-backprop` with the GitHub CLI and push. Private on purpose: it keeps the worked solution and the quiz answer key from being copied — graders use the live URL, not the source.
- The assignment brief and the raw original decks are kept out of version control (`.gitignore`) for academic-integrity reasons.

- Deploy on Vercel as a first-party Next.js app, imported from the repo through the dashboard.

10. A note on how I used these

I checked every prompt's output against the original lecture before keeping it. The no-text rule on the images is deliberate, so all the wording on the slides is written by me and can be verified. The quiz keys were re-checked by hand, not trusted as generated. Where the model got something wrong — mismatched icons, broken paths, drifting subtitles, the over-fast first narration — I changed the prompt or the settings and regenerated rather than accepting the first result. The deployment choices (no LFS, re-encode in-repo with a YouTube fallback kept available, private repo, brief excluded) were mine and made for stated reasons, not defaults the tool picked.